



Elasticsearch

高性能分布式搜索引擎



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌

Elasticsearch

黑马欢迎您! 请登录 免费注册

首页 | 我的购物车 | 我的黑马 | 黑马会员 | 企业采购 | 关注黑马 | 客户服务 | 网站导航

黑马商城 HMALL.COM

停用测试 ☒ 启用测试

电视

搜索

全部结果:

分类	手机	曲面电视	拉杆箱	休闲鞋	休闲鞋	硬盘	真皮包			
品牌	希捷	小米	华为	oppo	尤妮佳	莎米特	意尔康	新秀丽	Apple	锤子
价格	100以下	100~299元	300~599元	600~899元	900~1599元	1600以上元				

元素 控制台 源代码 网络 X 性能 内存 应用程序 安全性 Lighthouse CSS 概述 欢迎 Vue +

保留日志 禁用缓存 无限制

筛选器 ☐ 反转 ☐ 隐藏数据 URL 全部 Fetch/XHR JS CSS Img 媒体 字体 文档 WS Wasm 清单 其他 ☐ 已阻止 Cookie ☐ 已阻止请求 ☐ 第三方请求

2000 毫秒	4000 毫秒	6000 毫秒	8000 毫秒	10000 毫秒	12000 毫秒	14000 毫秒	16000 毫秒	18000 毫秒	20000 毫秒	22000 毫秒
---------	---------	---------	---------	----------	----------	----------	----------	----------	----------	----------

名称	状态	类型	发起程序	大小	时间	履行者	时间线
☐ list?key=%E7%94%B5%E8%A7%86&pageNo=1&pageSize=...	200	xhr	axios.min.js:8	9.2 kB	132 毫秒		
☐ list?key=%E7%94%B5%E8%A7%86&pageNo=1&pageSize=...	200	xhr	axios.min.js:8	9.2 kB	133 毫秒		
☐ list?key=%E7%94%B5%E8%A7%86&pageNo=1&pageSize=...	200	xhr	axios.min.js:8	9.2 kB	145 毫秒		
☐ list?key=%E7%94%B5%E8%A7%86&pageNo=1&pageSize=...	200	xhr	axios.min.js:8	9.2 kB	137 毫秒		
☐ list?key=%E7%94%B5%E8%A7%86&pageNo=1&pageSize=...	200	xhr	axios.min.js:8	9.2 kB	133 毫秒		

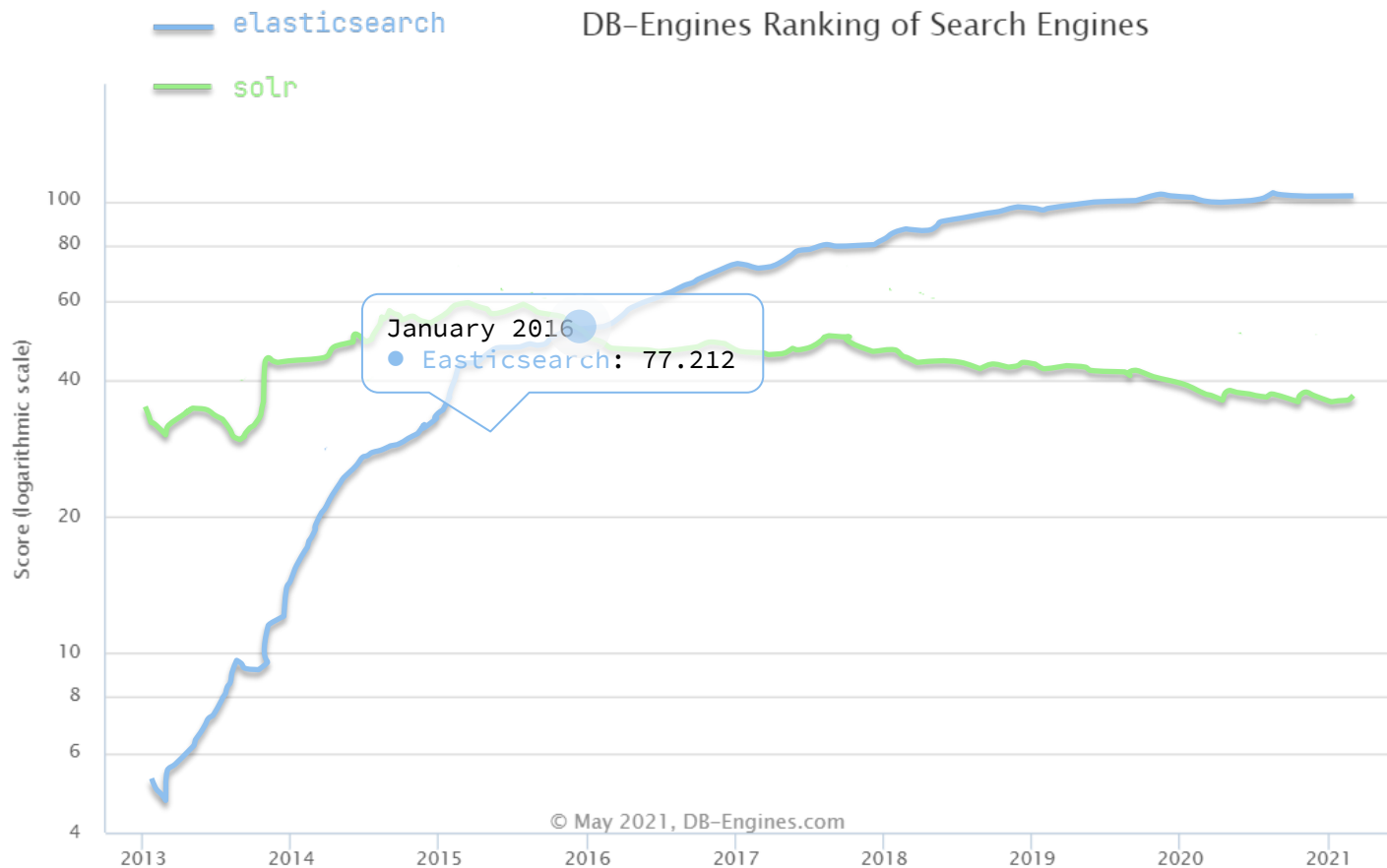
黑马程序员-研究院

Elasticsearch



Elasticsearch

- 搜索引擎技术排名：
 1. Elasticsearch: 开源的分布式搜索引擎
 2. Splunk: 商业项目
 3. Solr: Apache的开源搜索引擎





目录

Contents

- ◆ 初识elasticsearch
- ◆ 索引库操作
- ◆ 文档操作
- ◆ RestAPI



初识elasticsearch



目录

Contents

- ◆ 认识和安装
- ◆ 倒排索引
- ◆ IK分词器
- ◆ 基础概念

认识和安装

Lucene是一个Java语言的搜索引擎类库，是Apache公司的顶级项目，由DougCutting于1999年研发。官网地址：<https://lucene.apache.org/>。

Lucene的优势：

- 易扩展
- 高性能（基于倒排索引）

Doug



认识和安装

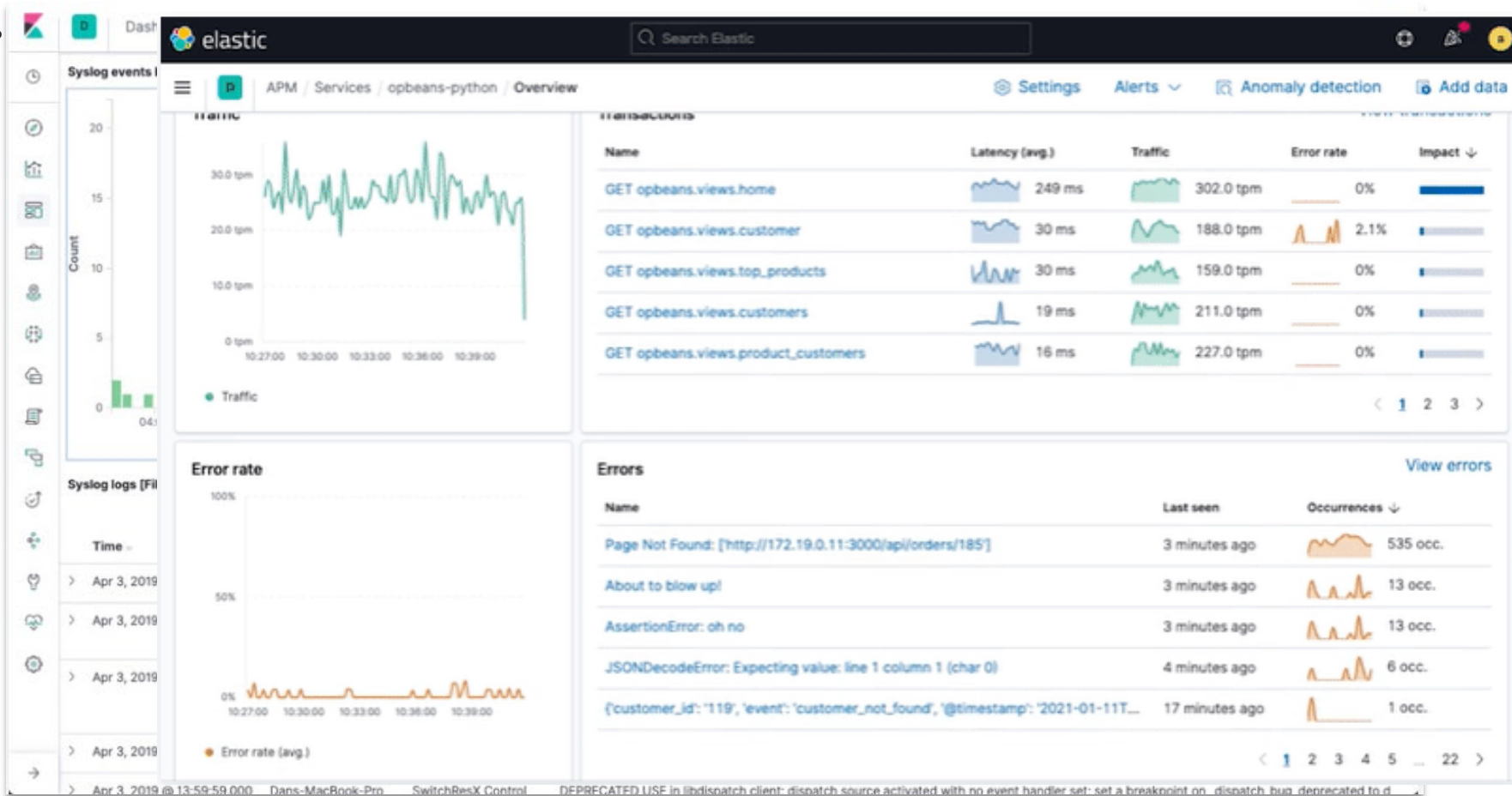
- 2004年Shay Banon基于Lucene开发了Compass
- 2010年Shay Banon 重写了Compass，取名为Elasticsearch。
- 官网地址: <https://www.elastic.co/cn/>，目前最新的版本是: 8.x.x
- elasticsearch具备下列优势:
 - 支持分布式，可水平扩展
 - 提供Restful接口，可被任何语言调用



elastic

认识和安装

elasticsearch结合kibana、Logstash、Beats，是一整套技术栈，被叫做ELK。被广泛应用在日志数据分析、实时监控等领域。



认识和安装

elasticsearch结合kibana、Logstash、Beats，是一整套技术栈，被叫做ELK。被广泛应用在日志数据分析、实时监控等领域。

数据可视化



Kibana

存储、计算
、搜索数据



Elasticsearch

数据抓取



Logstash



Beats



目录

Contents

- ◆ 认识 and 安装
- ◆ 倒排索引
- ◆ IK分词器
- ◆ 基础概念

倒排索引

传统数据库（如MySQL）采用正向索引，例如给下表（tb_goods）中的id创建索引：

id	title		price
1	小米手机		3499
2	华为手机	2 华为手机	4999
3	华为小 3	华为小米充电器	49
4	小米手环		299
...	...		

正向索引



倒排索引

elasticsearch采用倒排索引：

- 文档 (document) ： 每条数据就是一个文档
- 词条 (term) ： 文档按照语义分成的词语

id	title	price
1	小米手机	3499
2	华为手机	4999
3	华为小米充电器	49
4	小米手环	299



词条 (term)	文档id
	,2,4
	,2
	,3
	3
	4





总结

什么是文档和词条?

- 每一条数据就是一个文档
- 对文档中的内容分词，得到的词语就是词条

什么是正向索引?

- 基于文档id创建索引。根据id查询快，但是查询词条时必须先找到文档，而后判断是否包含词条

什么是倒排索引?

- 对文档内容分词，对词条创建索引，并记录词条所在文档的id。查询时先根据词条查询到文档id，而后根据文档id查询文档



目录

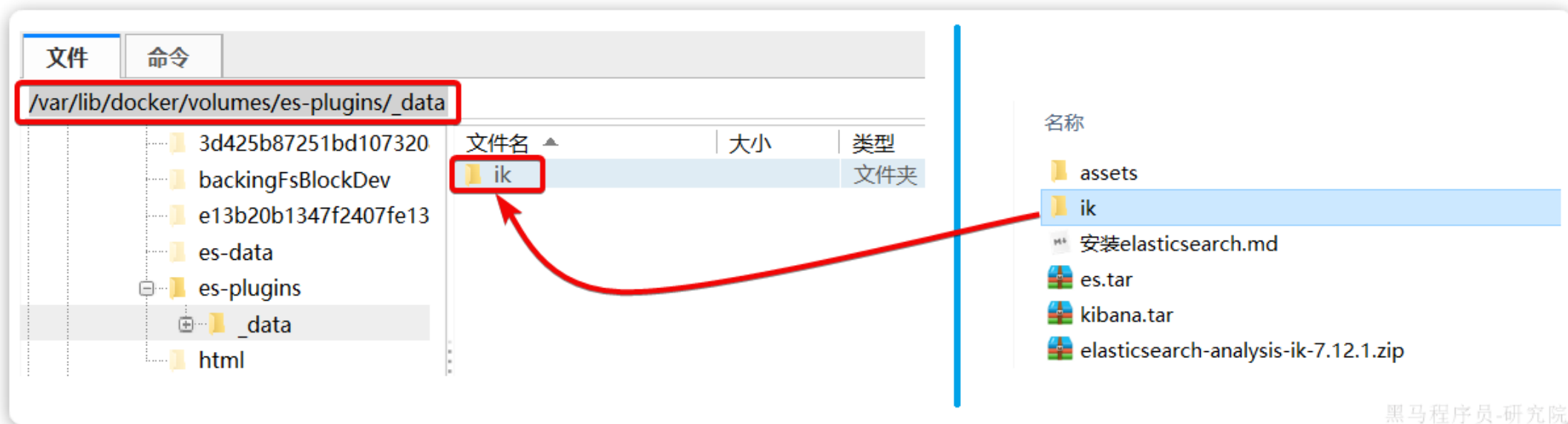
Contents

- ◆ 认识 and 安装
- ◆ 倒排索引
- ◆ IK分词器
- ◆ 基础概念

IK分词器

中文分词往往需要根据语义分析，比较复杂，这就需要用到中文分词器，例如**IK分词器**。IK分词器是林良益在2006年开源发布的，其采用的正向迭代最细粒度切分算法一直沿用至今。

其安装的方式也比较简单，只要将资料提供好的分词器放入elasticsearch的插件目录即可：



IK分词器

在Kibana的DevTools中可以使用下面的语法来测试IK分词器：

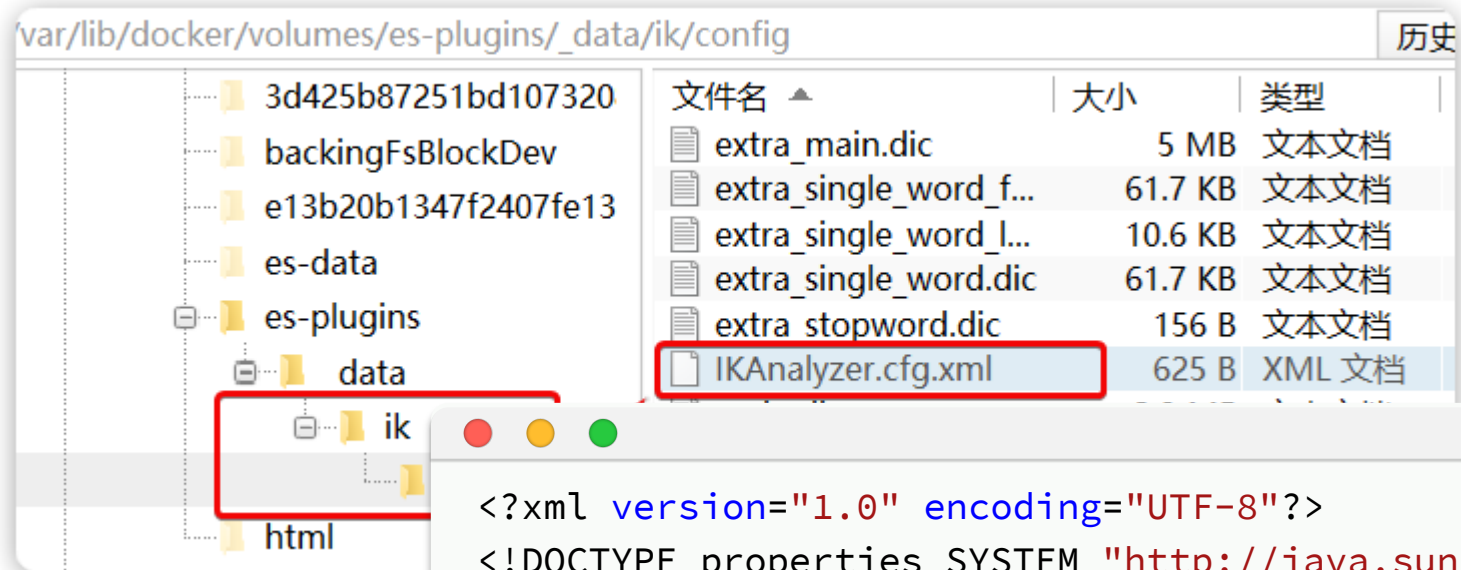
```
POST /_analyze
{
  "analyzer": "standard",
  "text": "黑马程序员学习java太棒了"
}
```

语法说明：

- ◆ POST：请求方式
- ◆ /_analyze：请求路径，这里省略了<http://192.168.150.101:9200>，有kibana帮我们补充
- ◆ 请求参数，json风格：
 - analyzer：分词器类型，这里是默认的standard分词器
 - text：要分词的内容

IK分词器

IK分词器允许我们配置拓展词典来增加自定义的词库：



```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE properties SYSTEM "http://java.sun.com/dtd/properties.dtd">
<properties>
  <comment>IK Analyzer 扩展配置</comment>
  <!--用户可以在这里配置自己的扩展字典 *** 添加扩展词典-->
  <entry key="ext_dict">ext.dic</entry>
</properties>
```



总结

分词器的作用是什么?

- 创建倒排索引时，对文档分词
- 用户搜索时，对输入的内容分词

IK分词器有几种模式?

- ik_smart: 智能切分，粗粒度
- ik_max_word: 最细切分，细粒度IK分词器

如何拓展分词器词库中的词条?

- 利用config目录的IkAnalyzer.cfg.xml文件添加拓展词典
- 在词典中添加拓展词条



目录

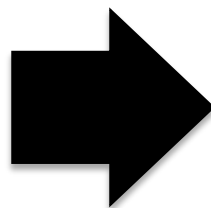
Contents

- ◆ 认识 and 安装
- ◆ 倒排索引
- ◆ IK分词器
- ◆ 基础概念

基础概念

elasticsearch中的文档数据会被序列化为json格式后存储在elasticsearch中。

id	title	price
1	小米手机	3499
2	华为手机	4999
3	华为小米充电器	49
4	小米手环	299



```
{
  "id": 1,
  "title": "小米手机",
  "price": 3499
}

{
  "id": 2,
  "title": "华为手机",
  "price": 4999
}

{
  "id": 3,
  "title": "华为小米充电器",
  "price": 49
}

{
  "id": 4,
  "title": "小米手环",
  "price": 299
}
```

基础概念

```
{  
  "id": 4,  
  "title": "小米手环",  
  "price": 299  
}
```

```
{  
  "id": 11,  
  "userId": 102,  
  "goodsId": 2,  
  "totalFee": 328  
}
```

```
{  
  "id": 102,  
  "name": "李四",  
  "age": 24  
}
```

```
{  
  "id": 10,  
  "userId": 101,  
  "goodsId": 1,  
  "totalFee": 294  
}
```

```
{  
  "id": 2,  
  "title": "华为手机",  
  "price": 4999  
}
```

```
{  
  "id": 1,  
  "title": "小米手机",  
  "price": 3499  
}
```

```
{  
  "id": 103,  
  "name": "麻子",  
  "age": 18  
}
```

```
{  
  "id": 101,  
  "name": "张三",  
  "age": 21  
}
```

基础概念

索引 (index) : 相同类型的文档的集合

映射 (mapping) : 索引中文档的字段约束信息, 类似表的结构约束

商品索引

```
{
  "id": 1,
  "title": "小米手机",
  "price": 3499
}

{
  "id": 2,
  "title": "华为手机",
  "price": 4999
}

{
  "id": 4,
  "title": "小米手环",
  "price": 299
}
```

用户索引

```
{
  "id": 101,
  "name": "张三",
  "age": 21
}

{
  "id": 102,
  "name": "李四",
  "age": 24
}

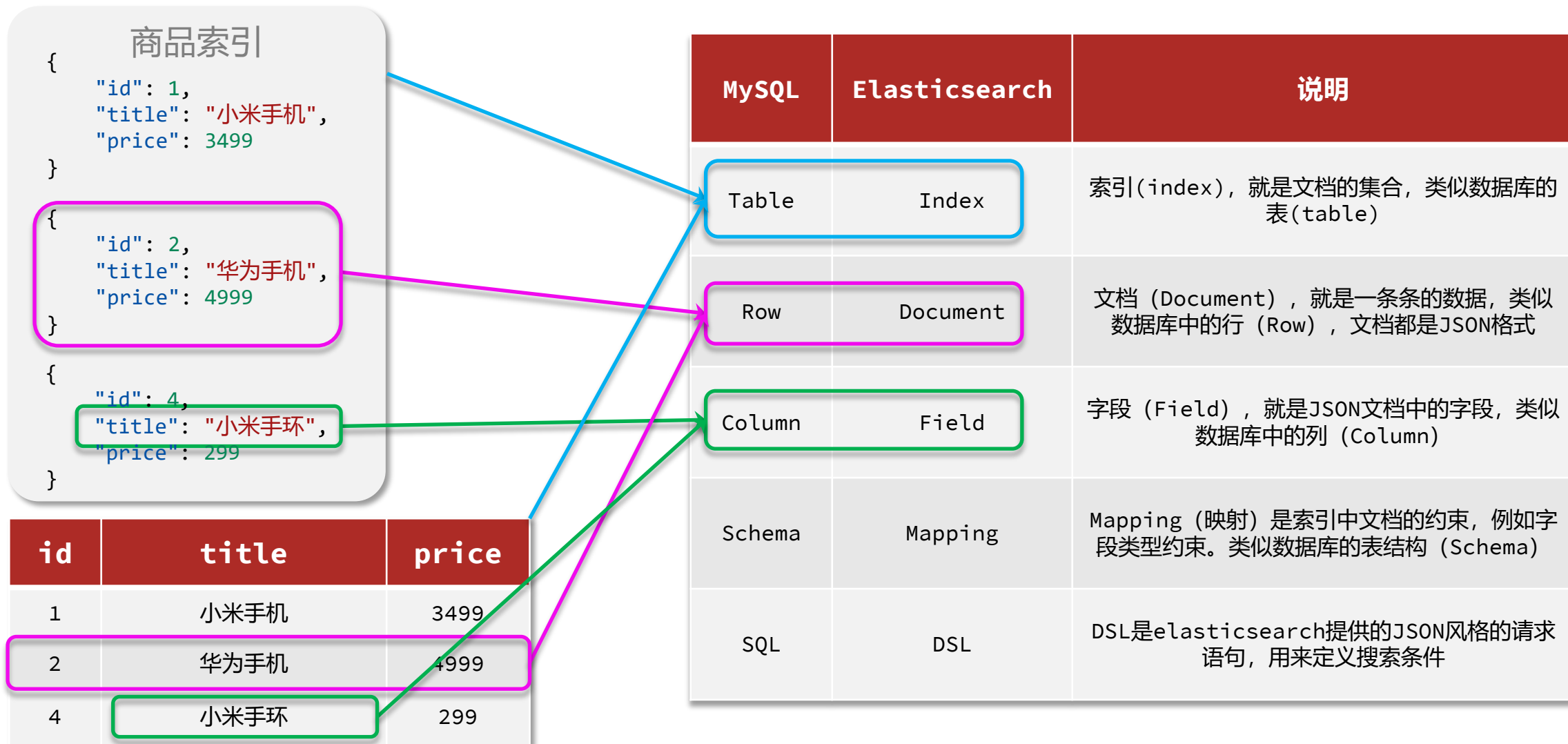
{
  "id": 103,
  "name": "麻子",
  "age": 18
}
```

订单索引

```
{
  "id": 10,
  "userId": 101,
  "goodsId": 1,
  "totalFee": 294
}

{
  "id": 11,
  "userId": 102,
  "goodsId": 2,
  "totalFee": 328
}
```

基础概念





索引库操作

索引库操作

商品索引

```
{
  "id": 1,
  "title": "小米手机",
  "price": 3499
}
{
  "id": 2,
  "title": "华为手机",
  "price": 4999
}
{
  "id": 4,
  "title": "小米手环",
  "price": 299
}
```

id	title	price
1	小米手机	3499
2	华为手机	4999
4	小米手环	299

MySQL	Elasticsearch	说明
Table	Index	索引(index), 就是文档的集合, 类似数据库的表(table)
Row	Document	文档(Document), 就是一条条的数据, 类似数据库中的行(Row), 文档都是JSON格式
Column	Field	字段(Field), 就是JSON文档中的字段, 类似数据库中的列(Column)
Schema	Mapping	Mapping(映射)是索引中文档的约束, 例如字段类型约束。类似数据库的表结构(Schema)
SQL	DSL	DSL是elasticsearch提供的JSON风格的请求语句, 用来操作elasticsearch, 实现CRUD



目录

Contents

- ◆ Mapping映射属性
- ◆ 索引库操作

Mapping映射属性

mapping是对索引库中文档的约束，常见的mapping属性包括：

- type: 字段数据类型，常见的简单类型有：
 - 字符串: text (可分词的文本)、keyword (精确值, 例如: 品牌、国家、ip地址)
 - 数值: long、integer、short、byte、double、float、
 - 布尔: boolean
 - 日期: date
 - 对象: object
- index: 是否创建索引，默认为true
- analyzer: 使用哪种分词器
- properties: 该字段的子字段

```
{
  "age": 21,
  "weight": 52.1,
  "isMarried": false,
  "info": "黑马程序员Java讲师",
  "email": "zy@itcast.cn",
  "score": [99.1, 99.5, 98.9],
  "name": {
    "firstName": "云",
    "lastName": "赵"
  }
}
```



目录

Contents

- ◆ Mapping映射属性
- ◆ 索引库操作

索引库操作

Elasticsearch提供的所有API都是Restful的接口，遵循Restful的基本规范：

接口类型	请求方式	请求路径	请求参数
查询用户	GET	/users/{id}	路径中的id
新增用户	POST	/users	json格式user对象
修改用户	PUT	/users/{id}	- 路径中的id - json格式对象
删除用户	DELETE	/users/{id}	路径中的id

索引库操作

创建索引库和mapping的请求语法如下：

```
PUT /索引库名称
{
  "mappings": {
    "properties": {
      "字段名": {
        "type": "text",
        "analyzer": "ik_smart"
      },
      "字段名2": {
        "type": "keyword",
        "index": "false"
      },
      "字段名3": {
        "properties": {
          "子字段": {
            "type": "keyword"
          }
        }
      },
      // ...略
    }
  }
}
```

示例

```
PUT /heima
{
  "mappings": {
    "properties": {
      "info": {
        "type": "text",
        "analyzer": "ik_smart"
      },
      "email": {
        "type": "keyword",
        "index": "false"
      },
      "name": {
        "properties": {
          "firstName": {
            "type": "keyword"
          }
        }
      },
      // ... 略
    }
  }
}
```

索引库操作

查看索引库语法：

```
GET /索引库名
```

示例：

```
GET /heima
```

删除索引库的语法：

```
DELETE /索引库名
```

示例：

```
DELETE /heima
```

索引库操作

索引库和mapping一旦创建无法修改，但是可以添加新的字段，语法如下：

```
PUT /索引库名/_mapping
{
  "properties": {
    "新字段名": {
      "type": "integer"
    }
  }
}
```

示例：

```
PUT /heima/_mapping
{
  "properties": {
    "age": {
      "type": "integer"
    }
  }
}
```



总结

索引库操作有哪些?

- 创建索引库: PUT /索引库名
- 查询索引库: GET /索引库名
- 删除索引库: DELETE /索引库名
- 添加字段: PUT /索引库名/_mapping



文档操作



目录

Contents

◆ 文档CRUD

◆ 批量处理

文档CRUD

新增文档的请求格式如下：

```
POST /索引库名/_doc/文档id
{
  "字段1": "值1",
  "字段2": "值2",
  "字段3": {
    "子属性1": "值3",
    "子属性2": "值4"
  },
  // ...
}
```

示例

```
POST /heima/_doc/1
{
  "info": "黑马程序员Java讲师",
  "email": "zy@itcast.cn",
  "name": {
    "firstName": "云",
    "lastName": "赵"
  }
}
```

文档CRUD

查看文档请求格式:

```
GET /索引库名/_doc/文档id
```

示例:

```
GET /heima/_doc/1
```

删除索引库的请求格式:

```
DELETE /索引库名/_doc/文档id
```

示例:

```
DELETE /heima/_doc/1
```

文档CRUD

方式一：全量修改，会删除旧文档，添加新文档

```
PUT /索引库名/_doc/文档id
{
  "字段1": "值1",
  "字段2": "值2",
  // ... 略
}
```

示例

```
PUT /heima/_doc/1
{
  "info": "黑马程序员高级Java讲师",
  "email": "zy@itcast.cn",
  "name": {
    "firstName": "云",
    "lastName": "赵"
  }
}
```

方式二：增量修改，修改指定字段值

```
POST /索引库名/_update/文档id
{
  "doc": {
    "字段名": "新的值",
  }
}
```

示例

```
POST /heima/_update/1
{
  "doc": {
    "email": "ZhaoYun@itcast.cn"
  }
}
```



总结

文档操作有哪些?

- 创建文档: POST /索引库名/_doc/文档id { json文档 }
- 查询文档: GET /索引库名/_doc/文档id
- 删除文档: DELETE /索引库名/_doc/文档id
- 修改文档:
 - 全量修改: PUT /索引库名/_doc/文档id { json文档 }
 - 增量修改: POST /索引库名/_update/文档id { "doc": {字段}}



目录

Contents

◆ 文档CRUD

◆ 批量处理

批量处理

Elasticsearch中允许通过一次请求中携带多次文档操作，也就是批量处理，语法格式如下：

```
POST /_bulk
{ "index" : { "_index" : "索引库名", "_id" : "1" } }
{ "字段1" : "值1", "字段2" : "值2" }
{ "index" : { "_index" : "索引库名", "_id" : "1" } }
{ "字段1" : "值1", "字段2" : "值2" }
{ "index" : { "_index" : "索引库名", "_id" : "1" } }
{ "字段1" : "值1", "字段2" : "值2" }
{ "delete" : { "_index" : "test", "_id" : "2" } }
{ "update" : { "_id" : "1", "_index" : "test" } }
{ "doc" : { "field2" : "value2" } }
```



JavaRestClient



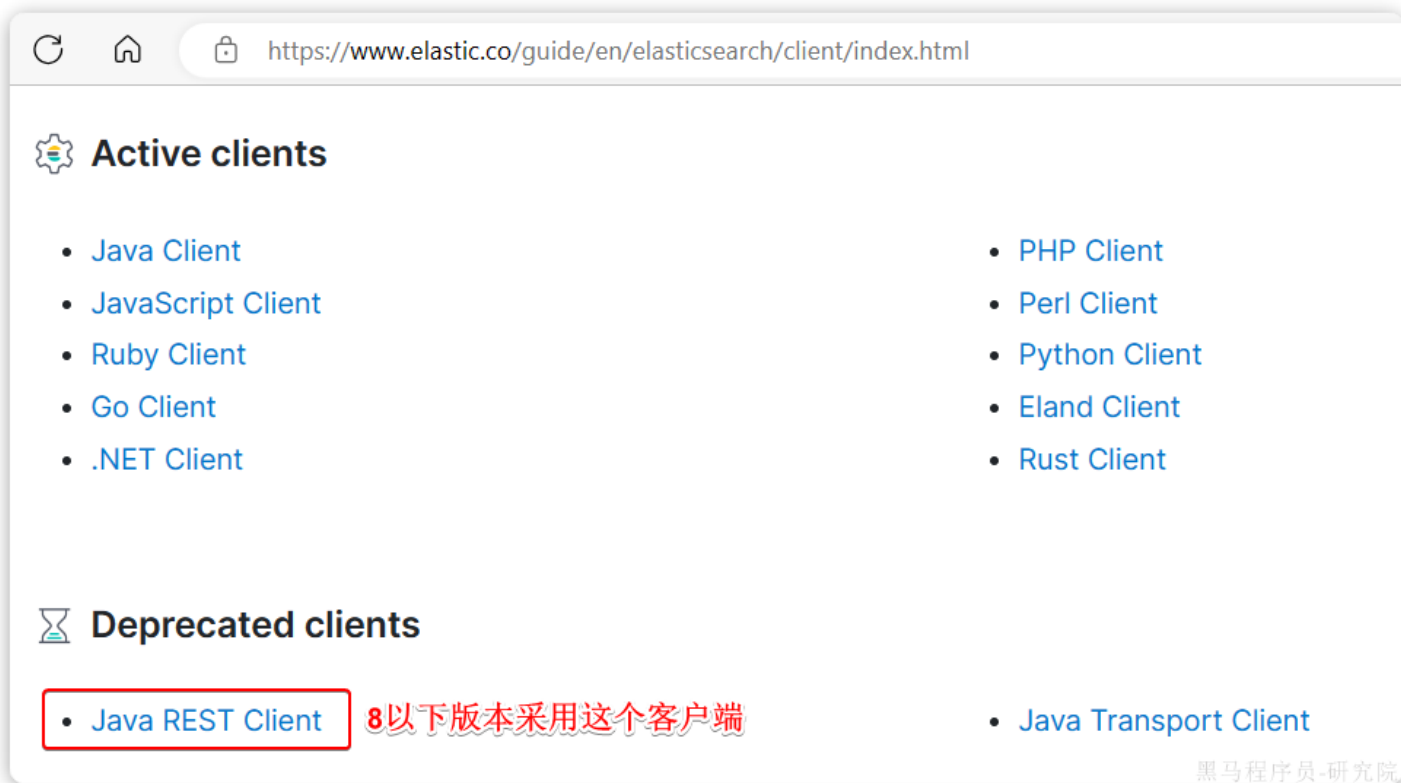
目录

Contents

- ◆ 客户端初始化
- ◆ 商品Mapping映射
- ◆ 索引库操作
- ◆ 文档操作
- ◆ 批处理

JavaRestClient

Elasticsearch目前最新版本是8.0，其Java客户端有很大变化。不过大多数企业使用的还是8以下版本，所以我们选择使用早期的JavaRestClient客户端来学习。官方文档地址：[Elasticsearch Clients | Elastic](https://www.elastic.co/guide/en/elasticsearch/client/index.html)



客户端初始化

1. 引入es的RestHighLevelClient依赖:

```
<dependency>  
  <groupId>org.elasticsearch.client</groupId>  
  <artifactId>elasticsearch-rest-high-level-client</artifactId>  
</dependency>
```

2. 因为SpringBoot默认的ES版本是7.17.0，所以我们需要覆盖默认的ES版本:

```
<properties>  
  <elasticsearch.version>7.12.1</elasticsearch.version>  
</properties>
```

3. 初始化RestHighLevelClient:

```
RestHighLevelClient client = new RestHighLevelClient(RestClient.builder(  
    HttpHost.create("http://192.168.150.101:9200")  
));
```



目录

Contents

- ◆ 客户端初始化
- ◆ 商品Mapping映射
- ◆ 索引库操作
- ◆ 文档操作
- ◆ 批处理

商品Mapping映射

我们要实现商品搜索，那么索引库的字段肯定要满足页面搜索的需求：

搜索字段

排序字段

展示字段



黑马商城
Hmall.com

全部结果:

分类

手机

曲面电视

拉杆箱

休闲鞋

硬盘

真皮包

品牌

希捷

小米

华为

oppo

尤妮佳

莎米特

价格

100以下

100~299元

300~599元

600~899元

900~1599元

1600以上

默认

销量

价格



¥ 137.00

HMDIME 牛仔背带裤女2018秋冬新品 韩版学院风潮流百搭弹力修身补

0 条评价

最近1个月销量: 0

立即购买

加入购物车



¥ 354.00

【官方正品 品牌直营】奥古狮登小白鞋女秋季透气2018新款百搭韩版

0 条评价

最近1个月销量: 0

立即购买

加入购物车



¥ 718.00

姬龙雪 guy laroche女包 包女牛皮大容量单肩包女

0 条评价

最近1个月销量: 0

立即购买

加入购物车

名称:	item				
注释:	商品表				
#	名称	数据类型	注释	是否需要	是否搜索
1	id	BIGINT	商品id	✓	✓
2	name	VARCHAR	SKU名称	✓	✓
3	price	INT	价格 (分)	✓	✓
4	stock	INT	库存数量		
5	image	VARCHAR	商品图片	✓	
6	category	VARCHAR	类目名称	✓	✓
7	brand	VARCHAR	品牌名称	✓	✓
8	spec	VARCHAR	规格		
9	sold	INT	销量	✓	✓
10	comment_count	INT	评论数	✓	
11	isAD	TINYINT	是否是推广广告	✓	✓
12	status	INT	商品状态 1-正常, 2-下架, 3-删除		
13	create_time	DATETIME	创建时间		
14	update_time	DATETIME	更新时间	✓	✓

高级软件人才培训专家



目录

Contents

- ◆ 客户端初始化
- ◆ 商品Mapping映射
- ◆ 索引库操作
- ◆ 文档操作
- ◆ 批处理

索引库操作

创建索引库的JavaAPI与Restful接口API对比:

@Test

void testCreateIndex() throws IOException {

// 1. 创建Request对象

CreateIndexRequest request = new CreateIndexRequest("items");

// 2. 请求参数, MAPPING_TEMPLATE是静态常量字符串, 内容是JSON格式请求体

request.source(MAPPING_TEMPLATE, XContentType.JSON);

// 3. 发起请求

client.indices().create(request, RequestOptions.DEFAULT);

}

返回的对象中包含
索引库操作的所有方法

PUT /items

```
{
  "mappings": {
    "properties": {
      "id": {
        "type": "keyword"
      },
      "name": {
        "type": "text",
        "analyzer": "ik_max_word"
      },
      "price": { },
      "stock": { },
      "image": { },
      "category": { },
      "brand": { },
      "sold": { },
      "commentCount": {
        "type": "integer"
      },
      "isAD": {
        "type": "boolean"
      },
      "updateTime": {
        "type": "date"
      }
    }
  }
}
```

索引库操作

删除索引库:

```
@Test
void testDeleteIndex() throws IOException {
    // 1. 创建Request对象
    DeleteIndexRequest request = new DeleteIndexRequest("indexName");
    // 2. 发起请求
    client.indices().delete(request, RequestOptions.DEFAULT);
}
```

查询索引库信息:

```
@Test
void testGetIndex() throws IOException {
    // 1. 创建Request对象
    GetIndexRequest request = new GetIndexRequest("indexName");
    // 2. 发起请求
    client.indices().get(request, RequestOptions.DEFAULT);
}
```



总结

索引库操作的基本步骤:

- ① 创建XxxIndexRequest。XXX是Create、Get、Delete
- ② 准备请求参数（Create时需要）
- ③ 发送请求。调用RestHighLevelClient#indices().xxx()方法，xxx是create、get(exists)、delete



目录

Contents

- ◆ 客户端初始化
- ◆ 商品Mapping映射
- ◆ 索引库操作
- ◆ 文档操作
- ◆ 批处理

文档操作

新增文档的JavaAPI如下:

```
@Test
void testIndexDocument() throws IOException {
    // 1. 创建request对象
    IndexRequest request = new IndexRequest("indexName").id("1");
    // 2. 准备JSON文档
    request.source("{\"name\": \"Jack\", \"age\": 21}", XContentType.JSON);
    // 3. 发送请求
    client.index(request, RequestOptions.DEFAULT);
}
```



文档操作

删除文档的JavaAPI如下：

```
@Test
void testDeleteDocumentById() throws IOException {
    // 1. 创建request对象
    DeleteRequest request = new DeleteRequest("indexName", "1");
    // 2. 删除文档
    client.delete(request, RequestOptions.DEFAULT);
}
```

```
DELETE /indexName/_doc/1
```

文档操作

查询文档包含查询和解析响应结果两部分。对应的JavaAPI如下：

```
@Test
void testGetDocumentById() throws IOException {
    // 1. 创建request对象
    GetRequest request = new GetRequest("indexName", "1");
    // 2. 发送请求，得到结果
    GetResponse response = client.get(request, RequestOptions.DEFAULT);
    // 3. 解析结果
    String json = response.getSourceAsString();

    System.out.println(json);
}
```

GET /indexName/_doc/1

```
{
  "_index" : "users",
  "_type" : "_doc",
  "_id" : "1",
  "_version" : 1,
  "_seq_no" : 0,
  "_primary_term" : 1,
  "found" : true,
  "_source" : {
    "name" : "Jack",
    "age" : 21
  }
}
```

文档操作

修改文档数据有两种方式：

- 方式一：全量更新。再次写入id一样的文档，就会删除旧文档，添加新文档。与新增的JavaAPI一致。
- 方式二：局部更新。只更新指定部分字段。

```
@Test
void testUpdateDocumentById() throws IOException {
    // 1. 创建request对象
    UpdateRequest request = new UpdateRequest("indexName", "1");
    // 2. 准备参数, 每2个参数为一对 key value
    request.doc(
        "age", 18,
        "name", "Rose"
    );
    // 3. 更新文档
    client.update(request, RequestOptions.DEFAULT);
}
```

```
POST /users/_update/1
{
  索引库名、 文档id
  "doc": {
    "name": "Rose",
    "age": 18
  }
}
```

要修改的字段



总结

文档操作的基本步骤:

- 初始化RestHighLevelClient
- 创建XxxRequest。XXX是Index、Get、Update、Delete
- 准备参数 (Index和Update时需要)
- 发送请求。调用RestHighLevelClient#.xxx()方法, xxx是index、get、update、delete
- 解析结果 (Get时需要)



目录

Contents

- ◆ 客户端初始化
- ◆ 商品Mapping映射
- ◆ 索引库操作
- ◆ 文档操作
- ◆ 批处理

批处理

```
POST /_bulk
{ "index" : { "_index" : "索引库名", "_id" : "1" } }
{ "字段1" : "值1", "字段2" : "值2" }

{ "index" : { "_index" : "索引库名", "_id" : "1" } }
{ "字段1" : "值1", "字段2" : "值2" }

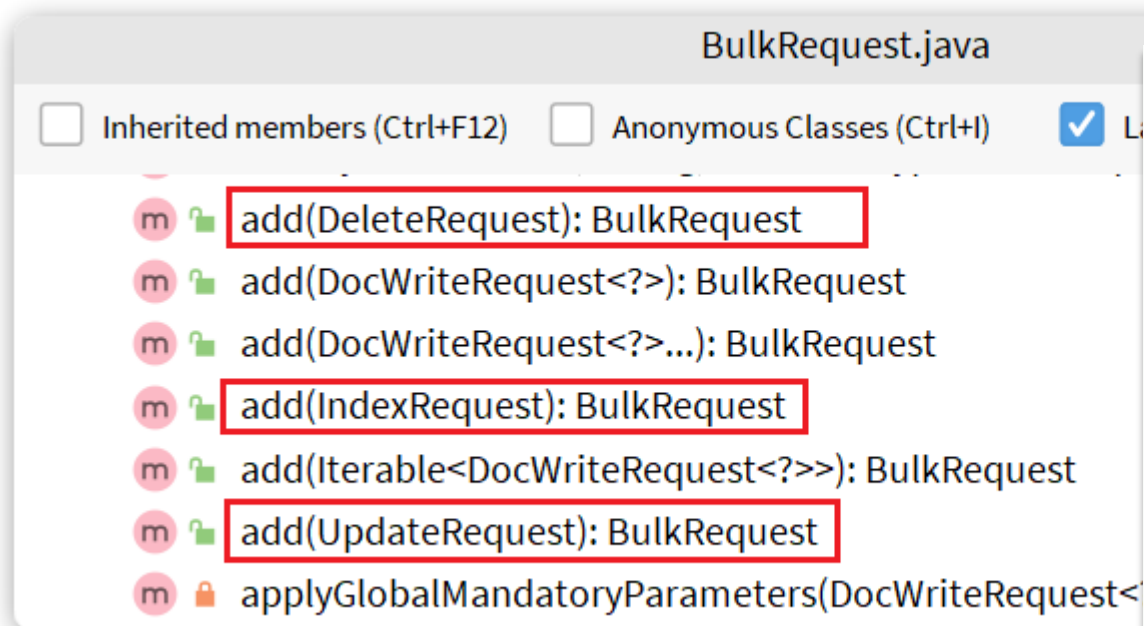
{ "index" : { "_index" : "索引库名", "_id" : "1" } }
{ "字段1" : "值1", "字段2" : "值2" }

{ "delete" : { "_index" : "test", "_id" : "2" } }

{ "update" : { "_id" : "1", "_index" : "test" } }
{ "doc" : { "field2" : "value2" } }
```

批处理

批处理代码流程与之前类似，只不过构建请求会用到一个名为BulkRequest来封装普通的CRUD请求：



```
POST /_bulk
{ "index" : { "_index" : "索引库名", "_id" : "1" } }
{ "字段1" : "值1", "字段2" : "值2" }

{ "index" : { "_index" : "索引库名", "_id" : "1" } }
{ "字段1" : "值1", "字段2" : "值2" }

{ "index" : { "_index" : "索引库名", "_id" : "1" } }
{ "字段1" : "值1", "字段2" : "值2" }

{ "delete" : { "_index" : "test", "_id" : "2" } }

{ "update" : { "_id" : "1", "_index" : "test" } }
{ "doc" : { "field2" : "value2" } }
```

批处理

批处理的API示例:

```
@Test
void testBulk() throws IOException {
    // 1. 创建Bulk请求
    BulkRequest request = new BulkRequest();
    // 2. 添加要批量提交的请求: 这里添加了两个新增文档的请求
    request.add(new IndexRequest("indexName")
        .id("101").source("json source", XContentType.JSON));
    request.add(new IndexRequest(" indexName ")
        .id("102").source("json source2", XContentType.JSON));
    // 3. 发起bulk请求
    client.bulk(request, RequestOptions.DEFAULT);
}
```



Elasticsearch

分布式搜索引擎



黑马程序员
www.itheima.com

传智教育旗下
高端IT教育品牌

Elasticsearch

Elasticsearch提供了基于JSON的DSL ([Domain Specific Language](#)) 语句以JSON格式来定义查询条件。类似这样：

```
Java POST _search
{
  "query": {
    "bool": {
      "must": {
        "term": { "user.id": "kimchy" }
      },
      "filter": {
        "term": { "tags": "production" }
      },
      "must_not": {
        "range": { "age": { "gte": 10, "lte": 20 } }
      },
      "should": [
        { "term": { "tags": "env1" } },
        { "term": { "tags": "deployed" } }
      ]
    }
  }
}
```



目录

Contents

- ◆ DSL查询
- ◆ Java客户端查询
- ◆ 数据聚合



DSL查询

DSL查询

Elasticsearch提供了DSL ([Domain Specific Language](#)) 查询，就是以JSON格式来定义查询条件。类似这样：

```
POST _search
{
  "query": {
    "bool": {
      "must": {
        "term": { "user.id" : "kimchy" }
      },
      "filter": {
        "term": { "tags" : "production" }
      },
      "must_not": {
        "range": { "age" : { "gte" : 10, "lte" : 20 } }
      },
      "should" : [
        { "term" : { "tags" : "env1" } },
        { "term" : { "tags" : "deployed" } }
      ]
    }
  }
}
```

DSL查询

DSL查询可以分为两大类：

- **叶子查询 (Leaf query clauses)**：一般是在特定的字段里查询特定值，属于简单查询，很少单独使用。
- **复合查询 (Compound query clauses)**：以逻辑方式组合多个叶子查询或者更改叶子查询的行为方式。

在查询以后，还可以对查询的结果做处理，包括：

- **排序**：按照1个或多个字段值做排序
- **分页**：根据from和size做分页，类似MySQL
- **高亮**：对搜索结果中的关键字添加特殊样式，使其更加醒目
- **聚合**：对搜索结果做数据统计以形成报表



目录

Contents

- ◆ 快速入门
- ◆ 叶子查询
- ◆ 复合查询
- ◆ 排序和分页
- ◆ 高亮显示

快速入门

基于DSL的查询语法如下：

```
GET /indexName/_search
{
  "query": {
    "查询类型": {
      "查询条件": "条件值"
    }
  }
}
```

```
// 查询所有
GET /indexName/_search
{
  "query": {
    "match_all": {
    }
  }
}
```



目录

Contents

- ◆ 快速入门
- ◆ 叶子查询
- ◆ 复合查询
- ◆ 排序和分页
- ◆ 高亮显示

叶子查询

叶子查询还可以进一步细分，常见的有：

- **全文检索 (full text) 查询**：利用分词器对用户输入内容分词，然后去词条列表中匹配。例如：
 - match_query
 - multi_match_query
- **精确查询**：不对用户输入内容分词，直接精确匹配，一般是查找keyword、数值、日期、布尔等类型。例如：
 - ids
 - range
 - term
- **地理 (geo) 查询**：用于搜索地理位置，搜索方式很多。例如：
 - geo_distance
 - geo_bounding_box

叶子查询

match查询：全文检索查询的一种，会对用户输入内容分词，然后去倒排索引库检索，语法：

```
GET /indexName/_search
{
  "query": {
    "match": {
      "FIELD": "TEXT"
    }
  }
}
```

multi_match：与match查询类似，只不过允许同时查询多个字段，语法：

```
GET /indexName/_search
{
  "query": {
    "multi_match": {
      "query": "TEXT",
      "fields": ["FIELD1", "FIELD12"]
    }
  }
}
```

叶子查询

精确查询，英文是Term-level query，顾名思义，词条级别的查询。也就是说不会对用户输入的搜索条件再分词，而是作为一个词条，与搜索的字段内容精确值匹配。

因此推荐查找keyword、数值、日期、boolean类型的字段。例如id、price、城市、地名、人名等作为一个整体才有含义的字段。

```
// term查询
GET /indexName/_search
{
  "query": {
    "term": {
      "FIELD": {
        "value": "VALUE"
      }
    }
  }
}
```

```
// range查询
GET /indexName/_search
{
  "query": {
    "range": {
      "FIELD": {
        "gte": 10,
        "lte": 20
      }
    }
  }
}
```



总结

match和multi_match的区别是什么?

- match: 根据一个字段查询
- multi_match: 根据多个字段查询, 参与查询字段越多, 查询性能越差

精确查询常见的有哪些?

- term查询: 根据词条精确匹配, 一般搜索keyword类型、数值类型、布尔类型、日期类型字段
- range查询: 根据数值范围查询, 可以是数值、日期的范围



目录

Contents

- ◆ 快速入门
- ◆ 叶子查询
- ◆ 复合查询
- ◆ 排序和分页
- ◆ 高亮显示

复合查询

复合查询大致可以分为两类：

- 第一类：基于逻辑运算组合叶子查询，实现组合条件，例如
 - ◆ bool
- 第二类：基于某种算法修改查询时的文档相关性算分，从而改变文档排名。例如：
 - ◆ function_score
 - ◆ dis_max

复合查询

布尔查询是一个或多个查询子句的组合。子查询的组合方式有：

- must: 必须匹配每个子查询，类似“与”
- should: 选择性匹配子查询，类似“或”
- must_not: 必须不匹配，不参与算分，类似“非”
- filter: 必须匹配，不参与算分

```
GET /items/_search
{
  "query": {
    "bool": {
      "must": [
        {"match": {"name": "手机"}}
      ],
      "should": [
```



全部结果：

分类	手机	曲面电视	拉杆箱	休闲鞋	休闲鞋	硬盘	真皮包			
品牌	希捷	小米	华为	oppo	尤妮佳	莎米特	意尔康	新秀丽	Apple	锤子
价格	100以下	100~299元	300~599元	600~899元	900~1599元	1600以上元				

案例

复合查询

需求：我们要搜索"智能手机"，但品牌必须是华为，价格必须是900~1599



目录

Contents

- ◆ 快速入门
- ◆ 叶子查询
- ◆ 复合查询
- ◆ 排序和分页
- ◆ 高亮显示

排序和分页

elasticsearch支持对搜索结果[排序](#)，默认是根据相关度算分（_score）来排序，也可以指定字段排序。可以排序字段类型有：keyword类型、数值类型、地理坐标类型、日期类型等。

```
GET /indexName/_search
{
  "query": {
    "match_all": {}
  },
  "sort": [
    {
      "FIELD": "desc" // 排序字段和排序方式ASC、DESC
    }
  ]
}
```

案例

实现排序功能

需求：搜索商品，按照销量排序，销量一样则按照价格升序

黑马商城
HMALL.COM

停用测试 ☒ 启用测试

搜索

全部结果:

分类	手机	曲面电视	拉杆箱	休闲鞋	休闲鞋	硬盘	真皮包			
品牌	希捷	小米	华为	oppo	尤妮佳	莎米特	意尔康	新秀丽	Apple	锤子
价格	100以下	100~299元	300~599元	600~899元	900~1599元	1600以上元				

默认 | 销量 ▾ | 价格 ▾ |

共 1294 件商品 1/65 < >



¥ 959.00



¥ 879.00



¥ 671.00



¥ 789.00



¥ 266.00

分页

elasticsearch 默认情况下只返回top10的数据。而如果要查询更多数据就需要修改分页参数了。elasticsearch中通过修改from、size参数来控制要返回的分页结果：

- from：从第几个文档开始
- size：总共查询几个文档

```
GET /items/_search
{
  "query": {
    "match_all": {}
  },
  "from": 0, // 分页开始的位置，默认为0
  "size": 10, // 期望获取的文档总数
  "sort": [
    {"price": "asc"}
  ]
}
```

案例

实现排序和分页功能

需求：搜索商品，查询出销量排名前10的商品，销量一样时按照价格升序

黑马商城
HMALL.COM

停用测试 ☒ 启用测试

搜索

全部结果:

分类	手机	曲面电视	拉杆箱	休闲鞋	休闲鞋	硬盘	真皮包			
品牌	希捷	小米	华为	oppo	尤妮佳	莎米特	意尔康	新秀丽	Apple	锤子
价格	100以下	100~299元	300~599元	600~899元	900~1599元	1600以上元				

默认 | 销量 ▾ | 价格 ▾ |

共 1294 件商品 1/65 < >



¥ 959.00



¥ 879.00



¥ 671.00



¥ 789.00



¥ 266.00

深度分页问题

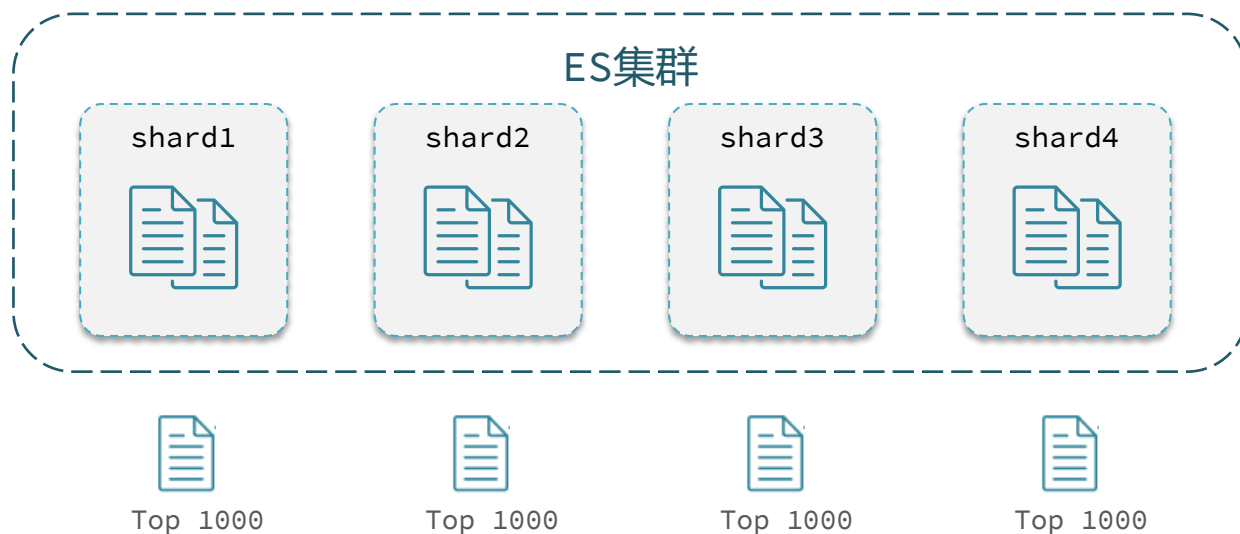
elasticsearch的数据一般会采用分片存储，也就是把一个索引中的数据分成N份，存储到不同节点上。查询数据时需要汇总各个分片的数据。

假如我们要查询第100页数据，每页查10条：

```
GET /hotel/_search
{
  "from": 990,
  "size": 10,
}
```

实现思路：

- ① 对数据排序
- ② 找出第990~1000名



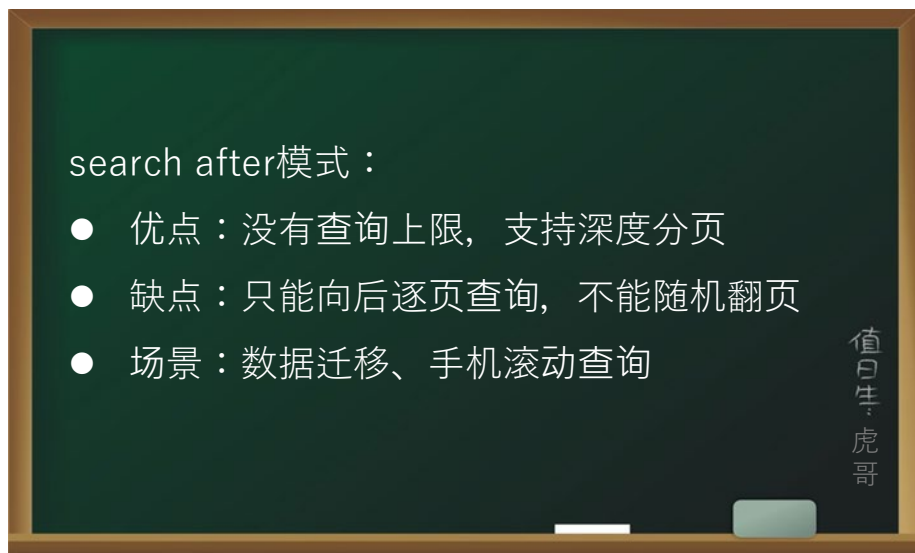
聚合所有结果，
重新排序选取前1000个

Top 1000

深度分页问题

针对深度分页，ES提供了两种解决方案，[官方文档](#)：

- search after：分页时需要排序，原理是从上一次的排序值开始，查询下一页数据。官方推荐使用的方式。
- scroll：原理将排序数据形成快照，保存在内存。官方已经不推荐使用。





目录

Contents

- ◆ 快速入门
- ◆ 叶子查询
- ◆ 复合查询
- ◆ 排序和分页
- ◆ 高亮显示

高亮显示

高亮显示：就是在搜索结果中把搜索关键字突出显示。



总结

搜索的完整语法:

```
GET /items/_search
{
  "query": {
    "match": {
      "name": "华为"
    }
  },
  "from": 0, // 分页开始的位置
  "size": 20, // 期望获取的文档总数
  "sort": [
    { "price": "asc" }, // 普通排序
  ],
  "highlight": {
    "fields": { // 高亮字段
      "name": {
        "pre_tags": "<em>", // 高亮字段的前置标签
        "post_tags": "</em>" // 高亮字段的后置标签
      }
    }
  }
}
```



JavaRestClient查询



目录

Contents

- ◆ 快速入门
- ◆ 构建查询条件
- ◆ 排序和分页
- ◆ 高亮显示

快速入门

数据搜索的Java代码我们分为两部分：

- 构建并发起请求
- 解析查询结果

```
@Test
```

```
void testMatchAll() throws IOException {
```

```
// 1. 准备Request
```

```
SearchRequest request = new SearchRequest(
```

```
// 2. 组织DSL参数
```

```
request.source()
```

```
.query(QueryBuilders.matchAllQuery
```

```
// 3. 发送请求，得到响应结果
```

```
SearchResponse response = client.search(
```

```
// ... 解析响应结果
```

```
}
```

```
SearchRequest request = new SearchRequest( ...indices: "items");  
request.source().
```

QueryBuilders

- matchAllQuery(): MatchAllQueryBuilder
- matchQuery(String, Object): MatchQueryBuilder
- idsQuery(): IdsQueryBuilder
- termQuery(String, String): TermQueryBuilder
- rangeQuery(String): RangeQueryBuilder
- wildcardQuery(String, String): WildcardQueryBuilder
- simpleQueryStringQuery(String): SimpleQueryStringBuilder
- boostingQuery(QueryBuilder, QueryBuilder): BoostingQueryBuilder
- boolQuery(): BoolQueryBuilder
- functionScoreQuery(QueryBuilder): FunctionScoreQueryBuilder
- geoDistanceQuery(String): GeoDistanceQueryBuilder
- geoBoundingBoxQuery(String): GeoBoundingBoxQueryBuilder

快速入门

解析查询结果的API:

```
@Test
void testMatchAll() throws IOException {
    // ... 略
    // 4. 解析结果
    SearchHits searchHits = response.getHits();
    // 4.1. 查询的总条数
    long total = searchHits.getTotalHits().value;
    // 4.2. 查询的结果数组
    SearchHit[] hits = searchHits.getHits();
    for (SearchHit hit : hits) {
        // 4.3. 得到source
        String json = hit.getSourceAsString();
        System.out.println(json);
    }
}
```

```
{
    "took" : 0,
    "timed_out" : false,
    "hits" : {
        "total" : {
            "value" : 2,
            "relation" : "eq"
        },
        "max_score" : 1.0,
        "hits" : [
            {
                "_index" : "heima",
                "_type" : "_doc",
                "_id" : "1",
                "_score" : 1.0,
                "_source" : {
                    "info" : "Java讲师",
                    "name" : "赵云",
                }
            },
            // ...
        ]
    }
}
```



总结

查询的基本步骤是：

1. 创建SearchRequest对象
2. 准备Request.source(), 也就是DSL。
 - ① QueryBuilders来构建查询条件
 - ② 传入Request.source() 的 query() 方法
3. 发送请求，得到结果
4. 解析结果（参考JSON结果，从外到内，逐层解析）



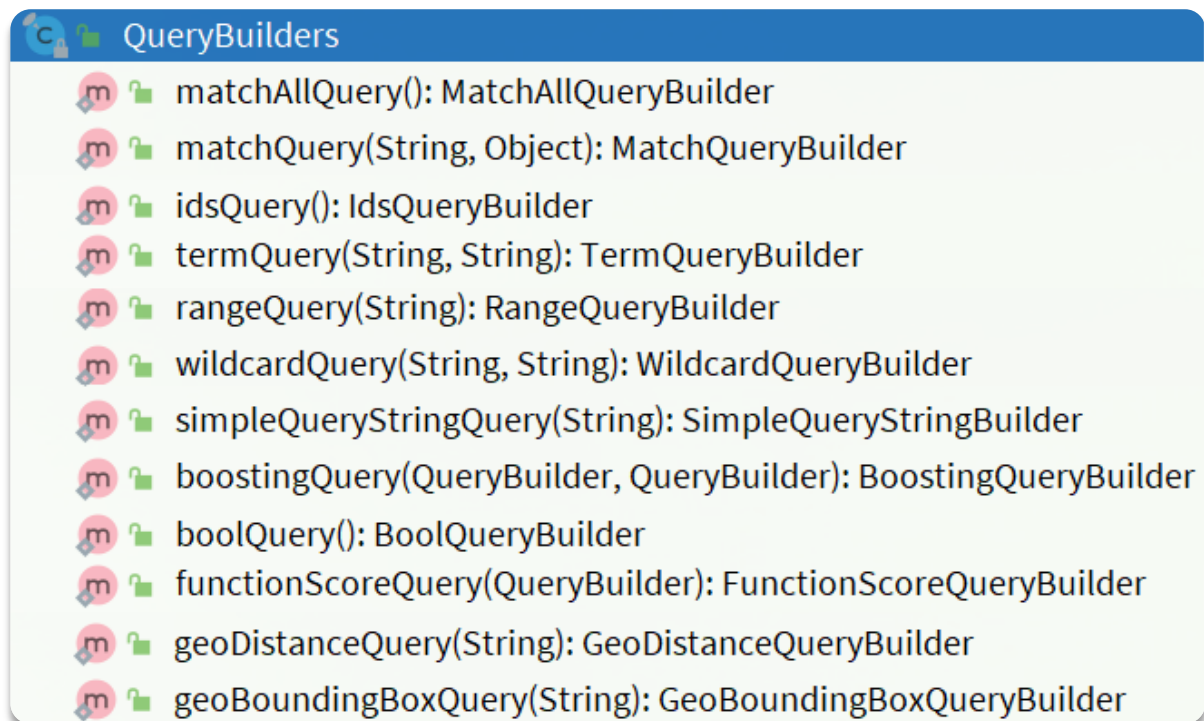
目录

Contents

- ◆ 快速入门
- ◆ 构建查询条件
- ◆ 排序和分页
- ◆ 高亮显示

构建查询条件

在JavaRestAPI中，所有类型的query查询条件都是由QueryBuilders来构建的：



构建查询条件

全文检索的查询条件构造API如下:

```
// 单字段查询
QueryBuilders.matchQuery("name", "脱脂牛奶");

// 多字段查询
QueryBuilders.multiMatchQuery("脱脂牛奶", "name", "category");
```

```
GET /items/_search
{
  "query": {
    "match": {
      "name": "脱脂牛奶"
    }
  }
}

GET /items/_search
{
  "query": {
    "multi_match": {
      "query": "脱脂牛奶",
      "fields": ["category", "name"]
    }
  }
}
```

构建查询条件

精确查询的查询条件构造API如下:

```
// 词条查询
QueryBuilders.termQuery("category", "牛奶");
// 范围查询
QueryBuilders.rangeQuery("price").gte(100).lte(150);
```

```
GET /items/_search
{
  "query": {
    "term": {
      "category": "牛奶"
    }
  }
}
GET /items/_search
{
  "query": {
    "range": {
      "price": { "gte": 100, "lte": 150 }
    }
  }
}
```

构建查询条件

布尔查询的查询条件构造API如下:

```
// 创建布尔查询
BoolQueryBuilder boolQuery = QueryBuilders.boolQuery();
// 添加must条件
boolQuery.must(
    QueryBuilders.termQuery("brand", "华为"));
// 添加filter条件
boolQuery.filter(
    QueryBuilders.rangeQuery("price").lte(2500));
```

```
GET /items/_search
{
  "query": {
    "bool": {
      "must": [
        {
          "term": { "brand": "华为" }
        }
      ],
      "filter": [
        {
          "range": {
            "price": { "lte": 2500 }
          }
        }
      ]
    }
  }
}
```

案例

构建复杂查询条件的搜索

需求：利用JavaRestClient实现搜索功能，条件如下：

- 搜索关键字为脱脂牛奶
- 品牌必须为德亚
- 价格必须低于300



目录

Contents

- ◆ 快速入门
- ◆ 构建查询条件
- ◆ 排序和分页
- ◆ 高亮显示

排序和分页

与query类似，排序和分页参数都是基于request.source()来设置：

```
// 查询
request.source().query(QueryBuilders.matchAllQuery());
// 分页
request.source().from(0).size(5);
// 价格排序
request.source().sort("price", SortOrder.ASC);
```

```
GET /indexName/_search
{
  "query": {
    "match_all": {}
  },
  "from": 0,
  "size": 5,
  "sort": [
    {
      "FIELD": "desc"
    },
  ]
}
```



目录

Contents

- ◆ 快速入门
- ◆ 构建查询条件
- ◆ 排序和分页
- ◆ 高亮显示

高亮显示

高亮显示的条件构造API如下:

```
request.source().highlighter(  
    SearchSourceBuilder.highlight()  
        .field("name")  
        .preTags("<em>")  
        .postTags("</em>")  
);
```

```
GET /items/_search  
{  
  "query": {  
    "match": {  
      "name": "脱脂牛奶"  
    }  
  },  
  "highlight": {  
    "fields": {  
      "name": {  
        "pre_tags": "<em>",  
        "post_tags": "</em>"  
      }  
    }  
  }  
}
```

高亮显示

高亮显示的结果解析API如下：

```
// 3.得到_source, 也就是原始json文档
String source = hit.getSourceAsString();
// 4.反序列化
ItemDoc item = JSONUtil.toBean(source, ItemDoc.class);
// 5.获取高亮结果
Map<String, HighlightField> hfs = hit.getHighlightFields();
if (CollUtils.isEmpty(hfs)) {
    // 5.1.有高亮结果, 获取name的高亮结果
    HighlightField hf = hfs.get("name");
    if (hf != null) {
        // 5.2.获取第一个高亮结果片段, 就是商品名称的高亮值
        String hfName = hf.getFragments()[0].string();
        item.setName(hfName);
    }
}
```

```
{
  "_index" : "items",
  "_type" : "_doc",
  "_id" : "33449279171",
  "_score" : 16.19757,
  "_source" : {
    "id" : "33449279171",
    "name" : "安佳脱脂牛奶",
    "price" : 3500,
    "stock" : 10000,
    "category" : "牛奶",
    "brand" : "葛兰纳诺",
    "sold" : 0,
    "commentCount" : 0,
    "isAD" : false
  },
  "highlight" : {
    "name" : [
      "安佳<em>脱脂</em><em>牛奶</em>"
    ]
  }
}
```



数据聚合

数据聚合

聚合 (aggregations) 可以实现对文档数据的统计、分析、运算。聚合常见的有三类：

- 桶 (Bucket) 聚合：用来对文档做分组
 - TermAggregation：按照文档字段值分组
 - Date Histogram：按照日期阶梯分组，例如一周为一组，或者一月为一组
- 度量 (Metric) 聚合：用以计算一些值，比如：最大值、最小值、平均值等
 - Avg：求平均值
 - Max：求最大值
 - Min：求最小值
 - Stats：同时求max、min、avg、sum等
- 管道 (pipeline) 聚合：其它聚合的结果为基础做聚合

注意

参与聚合的字段必须是Keyword、数值、日期、布尔的类型的字段





目录

Contents

- ◆ DSL聚合
- ◆ RestClient聚合

DSL聚合

我们要统计所有商品中共有哪些商品分类，其实就是以分类（category）字段对数据分组。category值一样的放在同一组，属于Bucket聚合中的Term聚合。

```
GET /items/_search
{
  "query": {"match_all": {}}, // 可以省略
  "size": 0, // 设置size为0, 结果中不包含文档, 只包含聚合结果
  "aggs": { // 定义聚合
    "cateAgg": { // 给聚合起个名字
      "terms": { // 聚合的类型, 按照品牌值聚合, 所以选择term
        "field": "category", // 参与聚合的字段
        "size": 20 // 希望获取的聚合结果数量
      }
    }
  }
}
```

DSL聚合

默认情况下，Bucket聚合是对索引库的所有文档做聚合，我们可以限定要聚合的文档范围，只要添加query条件即可。

```
GET /items/_search
{
  "query": {
    "bool": {
      "filter": [
        { "term": { "category": "手机" } },
        { "range": { "price": { "gte": 300000 } } }
      ]
    }
  },
  "size": 0,
  "aggs": {
    "brand_agg": {
      "terms": {
        "field": "brand",
        "size": 20
      }
    }
  }
}
```

DSL聚合

除了对数据分组（Bucket）以外，我们还可以对每个Bucket内的数据进一步做数据计算和统计。

例如：我想知道手机有哪些品牌，每个品牌的价格最小值、最大值、平均值。

```
GET /items/_search
{
  "query": {
    "term": {"category": "手机"}
  },
  "size": 0,
  "aggs": {
    "brand_agg": {
      "terms": {
        "field": "brand"
      },
      "aggs": {
        "price_stats": {
          "stats": {
            "field": "price"
          }
        }
      }
    }
  }
}
```



总结

aggs代表聚合，与query同级，此时query的作用是？

- 限定聚合的的文档范围

聚合必须的三要素：

- 聚合名称
- 聚合类型
- 聚合字段

聚合可配置属性有：

- size：指定聚合结果数量
- field：指定聚合字段



目录

Contents

- ◆ DSL聚合
- ◆ RestClient聚合

RestClient聚合

我们以品牌聚合为例：

```
request.source().size(0);  
request.source().aggregation(  
    AggregationBuilders  
        .terms("brand_agg")  
        .field("brand")  
        .size(20)  
);
```

```
GET /indexName/_search  
{  
  "size": 0,  
  "aggs": {  
    "brand_agg": {  
      "terms": {  
        "field": "brand",  
        "size": 20  
      }  
    }  
  }  
}
```

RestClient聚合

我们以品牌聚合为例：

// 解析聚合结果

```
Aggregations aggregations = response.getAggregations();
```

// 根据名称获取聚合结果

```
Terms brandTerms = aggregations.get("brand_agg");
```

// 获取桶

```
List<? extends Terms.Bucket> buckets = brandTerms.getBuckets();
```

// 遍历

```
for (Terms.Bucket bucket : buckets) {
```

// 获取key, 也就是品牌信息

```
String brandName = bucket.getKeyAsString();
```

```
System.out.println(brandName);
```

```
}
```

```
{
  "took" : 1,
  "timed_out" : false,
  "_shards" : {...},
  "hits" : {...},
  "aggregations" : {
    "brand_agg" : {
      "buckets" : [
        {
          "key" : "Apple",
          "doc_count" : 5
        },
        {
          "key" : "小米",
          "doc_count" : 1
        }
      ]
    }
  }
}
```